

Project Setup, Repo Structure & Execution Sequence

How dbt connects to Snowflake, what the project config controls, core CLI commands, and what actually happens when you run dbt.

profiles.yml — How dbt Connects to Snowflake

```
analytics:
  target: dev      # defines active output profile

  outputs:         # specify connections
    dev:
      type: snowflake
      account: abc123.eu-west-1
      user: jane@company.com
      role: <dbt_role>
      warehouse: <dbt_warehouse>    # default
      database: dbt_training         # default
      schema: <dbt_dev_jane>        # default
      threads: 4

  prod:
    ....
```

Working local

DO NOT store credentials and push it to Git.

Store your token in .secrets and .gitignore it.

Dev/ Prod schema rule

1. **Dev** is for development and testing.
2. You **don't write to prod** from your laptop — that's the orchestrator's job—
3. Create a **pull request**, get confirmation, merge your branch to main. The orchestrator write to prod.

Task: Write a `prod` profile, which stores in ``dbt_training.prod_your_name``. Check availability in dropdown.

dbt_project.yml — The Project Config

YAML — human-readable config. Indentation is everything: two spaces = one level.

```
name: dbt_training
version: "1.0.0"
profile: dbt_training          # must match the key in profiles.yml

model-paths: ["models"]      # one or multiple folders to look for models
source-paths: ["sources"]    # one or multiple folders to look for sources

models:
  dbt_training:              # ← project namespace — must match name above
    +persist_docs:           # writes comments and relationships to Snowflake
      relation: true
      columns: true
    staging:                 # must meet folder structure
      +materialized: view
      +tags: ["staging", "analytics", "finance"]  # used for selective execution,
                                                    # MANDATORY: layer and source and gold schema

  silver:
    +materialized: table
    +tags: ["silver", "analytics", "finance"]
```

+database and +schema — Where Models Land

+materialized controls *how* dbt builds a model. +database and +schema control *where* it lands in Snowflake.

```
models:
  analytics:
    silver:
      +materialized: table
      +database: SILVER    # Silver models always write to this database
      +schema: PUBLIC
    gold:
      +materialized: table
      +database: GOLD      # Gold models write to a separate database
      +schema: PUBLIC
```

Without +database

Every layer lands in whatever database `profiles.yml` specifies. No separation.

With +database

Each layer is routed to its own Snowflake database. The separation you see in Snowflake is enforced here.

Irritating default behaviour: +schema/ +database will take the default namespace and concat +schema. e.g. `database = dbt_training, schema = dbt_tweber, +schema = hubspot` creates `dbt_training.dbt_tweber_hubspot`.

Core CLI Commands

Command	What it does	When to use
<code>dbt compile</code>	Renders Jinja → raw SQL, no execution	Inspecting compiled output
<code>dbt run</code>	Executes models, no tests	In production – separate tests
<code>dbt test</code>	Runs tests only	Debugging one specific test
<code>dbt snapshot</code>	Creates snapshots	Update SCD2 models
<code>dbt build</code>	All of the above	Default in development
<code>dbt docs generate</code>	Builds doc site artifact	Regularly after new models

Remember: `dbt run` does NOT include snapshots. In production set up a separate task for `dbt snapshots` after bronze.

The Execution Sequence

What happens when you run `dbt run` or `dbt build`:

1. PARSE

Read all `.sql` and `.yaml` files, validate Jinja syntax

Fails here: Jinja syntax errors, missing macro definitions

2. RESOLVE

Build the DAG — resolve all `{{ ref() }}` and `{{ source() }}` calls

Fails here: circular refs, missing models

3. COMPILE

Render Jinja → raw SQL, write to `target/compiled/`

Fails here: undefined variables, bad config blocks

4. EXECUTE

Send compiled SQL to Snowflake

Fails here: SQL errors, permission errors, type mismatches

5. REPORT

Log results, write `manifest.json` and `run_results.json`

Always runs — even failed runs produce a report

LEGEND

- dbt only — no connection to the data warehouse
- First time connection to the data warehouse
- No effect on models — always runs regardless of outcome

Other Project Files You'll See

packages.yml

Declares external dbt packages (e.g. `dbt_utils`). Run `dbt deps` to install them. Use `dbt_utils` for surrogate key generation and date spines; `dbt_expectations` for extended test coverage.

seeds/

CSV files for small, static lookup tables that don't live in a source system — e.g. excluded test accounts, country-code mappings. Load with `dbt seed`.

analyses/

SQL files that use `ref()` and `source()` for lineage, but are never materialised. Useful for audit queries during a migration — compare old and new logic without polluting production models.

MODULE 02 COMPLETE

Next: Module 03

Jinja Basics for dbt

Prep Q1: What to be careful of when editing `profiles.yml` ?

Prep Q2: What does `dbt build` do that `dbt run` does not?

Prep Q3: At which phase does a Jinja syntax error appear?

Prep Q4: Read book pages 223-241!